

Almacenamiento en QEMU/KVM + libvirt

Pools, volúmenes, gestión y redimensión

✉ José Domingo Muñoz

🏫 IES Gonzalo Nazareno · Dos Hermanas

📖 IV · Infraestructura Virtual

01

Conceptos y tipos de almacenamiento

Pools, volúmenes y formatos de imagen

Conceptos básicos

POOL DE ALMACENAMIENTO

Recurso de almacenamiento gestionado por libvirt.

Tipos principales:

- `dir` — directorio del sistema de archivos
- `logical` — grupo de volúmenes LVM
- `netfs` — directorio NAS (NFS...)
- `iscsi` — disco desde servidor iSCSI

VOLUMEN

Medio de almacenamiento que representa el **disco de una MV**.

Según el tipo de pool puede ser:

- Un **fichero de imagen** (`dir` , `netfs`)
- Un **volumen lógico LVM** (`logical`)
- Un **disco iSCSI** (`iscsi`)

Tipo de pool: `dir`

- El pool controla un **directorio del host** (sistema de archivos)
- Los volúmenes son **ficheros de imagen de disco**:

FORMATO	CARACTERÍSTICAS
raw	Imagen binaria de disco. Ocupa todo el espacio asignado. Acceso más eficiente. Sin snapshots ni aprovisionamiento dinámico
qcow2	Formato QEMU copy-on-write. Solo ocupa el espacio de los datos (aprovisionamiento ligero). Soporta snapshots. Algo menos eficiente
vdi, vmdk	Formatos de otros sistemas de virtualización (VirtualBox, VMware)

 El tipo `dir` **no ofrece almacenamiento compartido** entre hosts.

Tipos de pool: **logical**, **netfs**, **iSCSI**

LOGICAL

- Controla un **Grupo de Volúmenes LVM**
- Los volúmenes son **LVs**
- Sin almacenamiento compartido
- Sin snapshots ni aprovisionamiento ligero

NETFS

- Monta un **directorío desde un servidor NAS** (NFS...)
- Los volúmenes son **ficheros de imagen**
- **Almacenamiento compartido** entre hosts

ISCSI

- Monta un **disco desde un servidor iSCSI**
- Los datos se guardan directamente en ese disco
- **Almacenamiento compartido** (con las consideraciones de acceso concurrente)

02

Gestión de pools y volúmenes

virsh, qemu-img y herramientas específicas

Dos enfoques para gestionar volúmenes

CON LIBVIRT (`VIRSH` / `VIRT-MANAGER`)

- Pool `dir` → crea una **imagen de disco** (qcow2, raw...)
- Pool `logical` → crea un **volumen lógico LVM**

Toda la gestión queda registrada en libvirt.

CON HERRAMIENTAS ESPECÍFICAS

- Pool `dir` → `qemu-img create ...`
→ después `pool-refresh`
- Pool `logical` → `lvcreate ...`
→ después `pool-refresh`

i Tras crear o modificar un volumen con herramientas externas, hay que ejecutar `virsh pool-refresh <pool>` para que libvirt lo detecte.

Gestión de pools de almacenamiento

```
virsh pool-list
virsh pool-info default
virsh pool-dumpxml default

# Crear, construir, arrancar y persistir un nuevo pool
virsh pool-define-as vm-images dir --target /srv/images
virsh pool-build      vm-images
virsh pool-start      vm-images
virsh pool-autostart  vm-images

# Detener y eliminar
virsh pool-destroy  vm-images
virsh pool-delete   vm-images
virsh pool-undefine vm-images
```


Gestión de volúmenes con libvirt

```
virsh vol-list default
virsh vol-list default --details
virsh vol-info prueba1.qcow2 default
virsh vol-dumpxml vol.qcow2 default

# Crear un volumen qcow2 de 10 GB en el pool default
virsh vol-create-as default vol1.qcow2 --format qcow2 10G

# Eliminar un volumen
virsh vol-delete vol1.qcow2 default
```

Gestión de volúmenes con `qemu-img`

Trabajamos con un pool de tipo `dir`:

```
cd /var/lib/libvirt/images

# Crear imagen qcow2 de 2 GB
qemu-img create -f qcow2 vol2.qcow2 2G

# Ver información de la imagen
qemu-img info vol2.qcow2

# Hacer que libvirt detecte el nuevo fichero
virsh pool-refresh vm-images
```

Creación de MV usando volúmenes existentes

```
virt-install \
    --virt-type kvm \
    --name prueba4 \
    --cdrom ~/iso/debian-11.3.0-amd64-netinst.iso \
    --os-variant debian10 \
    --disk vol=default/vol1.qcow2 \
    --memory 1024 \
    --vcpus 1
```

Otras formas de indicar el disco:

```
--disk path=/var/lib/libvirt/images/vol1.qcow2
--disk pool=vm-images,size=10
```

03

Añadir y redimensionar discos

Attach, resize y virt-resize

Añadir nuevos discos a una MV

```
# Añadir disco (funciona también en caliente)
virsh attach-disk prueba4 \
    /srv/images/vol2.qcow2 vdb \
    --driver=qemu --type disk \
    --subdriver qcow2          \
    --persistent

# Eliminar disco
virsh detach-disk prueba4 vdb --persistent
```



`--persistent` guarda el cambio en la configuración XML de la MV para que persista tras reinicios.

Redimensión de discos

Con la MV **parada**:

```
# Con libvirt
virsh vol-resize vol2.qcow2 3G --pool vm-images

# Con qemu-img
sudo qemu-img resize /srv/images/vol2.qcow2 3G
```

Con la MV **en ejecución** (en caliente):

```
virsh domblklist prueba4
virsh blockresize prueba4 /srv/images/vol2.qcow2 3G
```

Después, dentro de la MV, redimensionar el sistema de ficheros:

Redimensión del SF sin entrar en la MV

Usamos `virt-resize` para expandir el sistema de ficheros desde el host:

```
# 1. Ampliar el fichero de imagen
qemu-img resize vol1.qcow2 10G

# 2. Copiar la imagen (virt-resize trabaja origen → destino)
cp vol1.qcow2 newvol1.qcow2

# 3. Expandir la partición dentro de la imagen
virt-resize --expand /dev/sda1 vol1.qcow2 newvol1.qcow2

# 4. Reemplazar la imagen original
mv newvol1.qcow2 vol1.qcow2
```



`virt-resize` requiere el paquete `libguestfs-tools` y **siempre trabaja sobre una copia** del fichero original.

04

Clonación e instantáneas

Plantillas, clonación completa, enlazada y snapshots

Métodos de clonación

A PARTIR DE UNA MV

- `virt-clone` O `virt-manager`
- La MV clonada es **idéntica** a la original
- Problema: misma configuración, hostname, claves SSH...

A PARTIR DE UNA PLANTILLA

Imagen preconfigurada y generalizada (**copia maestra**):

- **Clonación completa (Full)**: copia completa de la plantilla. Requiere el mismo espacio en disco.
- **Clonación enlazada (Linked)**: la plantilla actúa como imagen base de solo lectura (*backing store*). Requiere mucho menos espacio.

Clonación desde una MV con `virt-clone`

```
# Clonación automática (nombre e imagen generados automáticamente)
virt-clone \
    --original prueba4 --auto-clone

# Especificando nombre e imagen destino
virt-clone \
    --original prueba4 \
    --name prueba5 \
    --file /var/lib/libvirt/images/prueba5.qcow2 \
    --auto-clone
```

 La MV clonada es **igual a la original**: mismo hostname, mismas claves SSH, misma configuración. Hay que entrar y modificarla manualmente.

Creación de una plantilla de MV

Una plantilla es una imagen **generalizada** a partir de la cual se crean nuevas MV no idénticas a la original.

1. Instalar y configurar una MV con todo el software necesario.
2. **Generalizar** la imagen con la MV parada:

```
sudo virt-sysprep -d prueba1 --hostname plantilla-debian11
```

3. **Proteger** la imagen para evitar arrancas la plantilla por error:

```
chmod -w prueba1.qcow2  
virsh domrename prueba1 plantilla-prueba1
```

Clonación completa de plantilla

```
virt-clone \  
    --original plantilla-prueba1 \  
    --name clone1 \  
    --auto-clone
```

También se puede usar `--file` para indicar el nombre de la nueva imagen.

Tras clonar, la MV no tiene claves SSH (se borraron al generalizar). Hay que entrar y regenerarlas:

```
echo "clone1" > /etc/hostname  
ssh-keygen -A  
reboot
```

Clonación enlazada (*Linked Clone*)

- La imagen de la MV clonada usa la de la plantilla como **imagen base** (*backing store*) en **solo lectura**
- La nueva imagen **solo almacena los cambios** respecto a la base
- Requiere **mucho menos espacio**, pero depende de la imagen base para funcionar

PASO 1

Crear el nuevo volumen indicando la imagen base (*backing store*)

PASO 2

Crear la nueva MV a partir del volumen, **sin instalar** SO (`--import`)

Paso 1 — Crear volumen con *backing store*

Comprobamos el tamaño de la imagen de la plantilla:

```
virsh domblkinfo plantilla-prueba1 vda --human
```

Creamos la nueva imagen con **virsh**:

```
virsh vol-create-as default clone2.qcow2 10G \  
  --format qcow2 \  
  --backing-vol      prueba1.qcow2 \  
  --backing-vol-format qcow2
```

O con **qemu-img**:

```
cd /var/lib/libvirt/images  
sudo qemu-img create -f qcow2 -b prueba1.qcow2 -F qcow2 clone2.qcow2 10G
```

Paso 1 — Verificar la imagen base

Con **virsh**:

```
virsh vol-dumpxml clone2.qcow2 default
...
<backingStore>
  <path>/var/lib/libvirt/images/prueba1.qcow2</path>
  <format type='qcow2' />
  ...
```

Con **qemu-img**:

```
sudo qemu-img info /var/lib/libvirt/images/clone2.qcow2
...
backing file: prueba1.qcow2
backing file format: qcow2
...
```

Paso 2 — Crear la MV enlazada

Con `virt-install` usando `--import` (no hay instalación, el disco ya tiene SO):

```
virt-install \  
    --virt-type kvm \  
    --name nueva_prueba \  
    --os-variant debian10 \  
    --disk path=/var/lib/libvirt/images/clone2.qcow2 \  
    --memory 1024 \  
    --vcpus 1 \  
    --import
```

Con `virt-clone` usando `--preserve-data` (no copia el volumen, lo reutiliza):

```
virt-clone \  
    --original plantilla-prueba1 \  
    --name clone2 \  
    --file /var/lib/libvirt/images/clone2.qcow2 \  
    --
```


Instantáneas de MV (*snapshots*)

- Guardan el **estado del disco y de la memoria** en un momento dado
- Permiten **volver a un estado anterior**
- Requieren imagen de disco en formato `qcow2`
- Se pueden hacer con la MV **apagada o encendida**

```
# Crear instantánea
virsh snapshot-create-as prueba2 \
    --name "instantánea1" \
    --description "Creada carpeta importante" \
    --atomic

# Listar y obtener info
virsh snapshot-list prueba2
sudo qemu-img info /var/lib/libvirt/images/prueba2.qcow2

# Restaurar
virsh snapshot-revert prueba2 instantánea1
```

IV · ALMACENAMIENTO EN QEMU/KVM + LIBVIRT

¡Gracias!

Almacenamiento en QEMU/KVM + libvirt

 José Domingo Muñoz

 IES Gonzalo Nazareno · Dos Hermanas

 IV